

×



Screen Object Pattern avec MobileWright

Guide technique d'intégration au framework, ligne par ligne

Document de référence pour l'intégration des testeurs logiciel DXC au framework d'automatisation mobile MobileWright.

Fil conducteur : automatisation d'un scénario YouTube sur Android

Rédigé par

ELMATTAD Zouhair — Testeur Logiciel

Version française — 12 août 2026

Document interne — Confidentiel. Propriété de DXC Technology. Réservé à un usage interne ; toute diffusion en dehors de l'entreprise est interdite sans autorisation.

© 2026 DXC Technology — Tous droits réservés.

Table des matières

1 Avant de commencer **3**

2 Ce que le pattern doit garantir **4**

3 Vue d'ensemble avant d'entrer dans les détails **4**

4 Identifier les écrans et définir la structure du dépôt **5**

5 ScreenBase : factoriser les fonctionnalités communes à tous les écrans **7**

5.1 L'import : ne récupérer que des contrats de types 7

5.2 Le champ protégé : visibilité et immutabilité 8

5.3 Le constructeur : recevoir la connexion plutôt que la fabriquer . . . 8

5.4 L'héritage : ce que chaque écran reçoit gratuitement 9

6 Les *Locators* : isoler la question « Où se trouve l'élément ? » **9**

6.1 La déclaration du champ : pourquoi `public` 9

6.2 Le constructeur : une syntaxe raccourcie propre à TypeScript 10

6.3 Un choix assumé : l'initialisation immédiate du locator 10

6.4 Le même principe, répété pour YouTube 10

7 Les classes *Screen* : transformer des opérations techniques en actions métier **10**

7.1 Trois imports, trois usages 11

7.2 L'héritage et le constructeur 12

7.3 Le locator privé : un détail d'implémentation caché 12

7.4 Des méthodes nommées par intention 12

7.5 YouTubeScreen : le même principe, une nuance métier 12

8 ScreenManager : le chef d'orchestre, expliqué ligne par ligne **14**

8.1 Les imports : trois lignes, trois natures différentes 14

8.2 La déclaration de la classe et son export 14

8.3 Les champs de la classe : la visibilité expliquée par un contraste réel 15

8.4 Le constructeur : le moment où trois objets prennent vie 15

8.5 Les méthodes d'accès : pourquoi retourner plutôt que recréer 16

9 Fixtures : brancher le framework sur le moteur de test **17**

9.1 Le type `Fixtures` : décrire une ressource avant de la fournir 17

9.2 Étendre le système de test de base 17

9.3 Ce que fait exactement la fonction fournie 18

9.4 Le rôle précis de `use(...)` 18

10 Le scénario : la preuve concrète du bénéfice **18**

10.1 Ce que ce fichier ne fait jamais 19

10.2 Ce que ce fichier raconte 19

10.3 La preuve de maintenabilité 19

11 En résumé **19**

1 Avant de commencer

Ce document explique en profondeur comment est construit le framework d'automatisation mobile utilisé dans ce projet et, surtout, **pourquoi** chaque décision de conception a été prise. Pour rendre les explications concrètes, nous nous appuierons sur un scénario réel : ouvrir YouTube depuis l'écran d'accueil d'un téléphone Android, rechercher une vidéo, puis interagir avec celle-ci.

Si vous débutez en programmation, retenez une idée essentielle avant de poursuivre votre lecture : une application mobile est constituée d'une succession d'**écrans** (écran d'accueil, YouTube, paramètres, etc.), et chaque écran contient des **éléments** avec lesquels l'utilisateur peut interagir, comme un bouton, un champ de recherche ou une vidéo. Automatiser un test consiste simplement à reproduire, à l'aide du code, les actions qu'un utilisateur effectuerait manuellement sur ces différents écrans.

Le **Screen Object Pattern** est une approche d'organisation du code qui associe chaque écran à un fichier dédié et chaque interaction à une méthode clairement définie. Cette structure rend le framework plus lisible, plus facile à maintenir et plus simple à faire évoluer.

Le reste de ce document part des concepts les plus fondamentaux et construit progressivement le framework, fichier par fichier. Une attention particulière sera portée au fichier central `screenManager.ts`, qui constitue le cœur de l'architecture. Nous y expliquerons en détail les principales décisions techniques, notamment la visibilité des attributs, le rôle du constructeur, les mécanismes d'exportation ainsi que la création et la gestion des objets.

À QUI S'ADRESSE CE DOCUMENT ?

Ce document s'adresse en priorité aux testeurs logiciel qui découvrent l'automatisation mobile et le langage TypeScript. Aucune connaissance préalable de la programmation orientée objet n'est requise : chaque notion (classe, objet, héritage, encapsulation...) est introduite au moment précis où elle apparaît dans le code, avec sa raison d'être. Des bases très générales en JavaScript ou TypeScript (variables, fonctions, importations) faciliteront la lecture, sans être indispensables.

CLASSE, OBJET, INSTANCE : TROIS MOTS POUR UNE SEULE IDÉE

Avant d'aller plus loin, une clarification de vocabulaire évite bien des confusions. Une **classe** est un plan : elle décrit la forme d'une chose (ses champs, ses méthodes), mais elle n'existe pas concrètement tant que personne ne l'utilise avec le mot-clé `new`. Un **objet** est la chose réellement construite à partir de ce plan. Chaque objet ainsi créé est appelé une **instance** de sa classe. Dans ce document, lorsque nous écrivons qu'un constructeur « crée un objet », cela signifie toujours : une instance de cette classe vient d'être fabriquée en mémoire, prête à être utilisée.

2 Ce que le pattern doit garantir

L'objectif d'un framework d'automatisation ne se limite pas à faire fonctionner un test une seule fois. Il doit avant tout être conçu pour résister aux évolutions de l'application au fil du temps. En effet, une application mobile évolue continuellement : de nouveaux écrans apparaissent, des boutons changent de position, des identifiants sont modifiés et de nouvelles fonctionnalités sont ajoutées à chaque nouvelle version. Le framework doit donc être capable de s'adapter à ces changements tout en limitant les modifications nécessaires.

Pour atteindre cet objectif, trois qualités fondamentales sont recherchées.

Lisibilité — Un scénario de test doit pouvoir se lire presque comme une phrase en langage naturel. Par exemple, l'instruction `screenManager.onHomeScreen().clickOnHomeButton()` permet de comprendre immédiatement que le test clique sur le bouton *Home*, sans qu'il soit nécessaire de connaître les détails techniques de cette interaction.

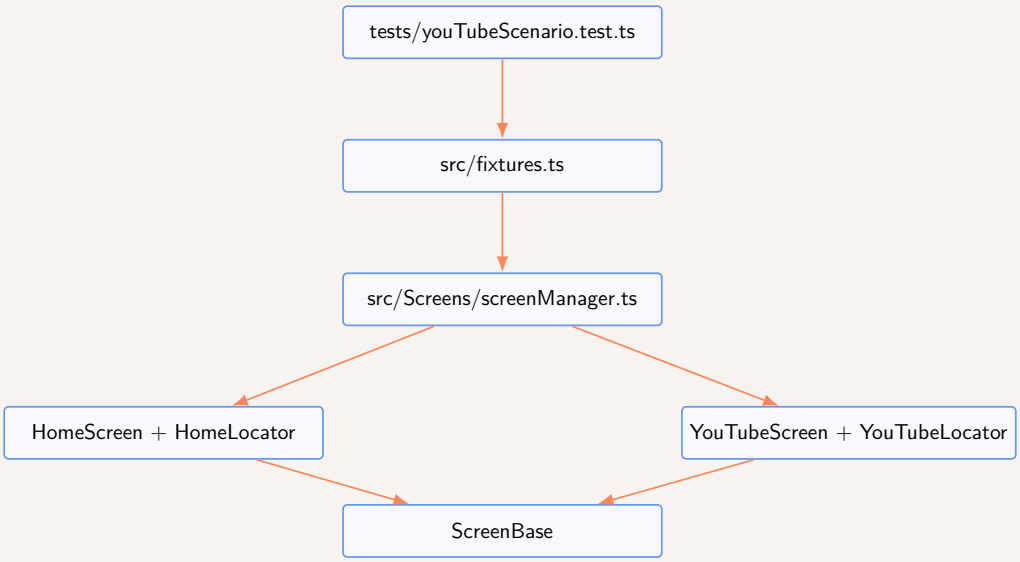
Maintenabilité — Lorsqu'un élément de l'application change, par exemple l'identifiant d'un bouton, la modification doit être effectuée à un seul endroit : dans le fichier contenant le *locator* correspondant. Les scénarios de test, quant à eux, restent inchangés, ce qui réduit considérablement les efforts de maintenance.

Scalabilité — Le framework doit pouvoir évoluer facilement. L'ajout d'un nouvel écran ou d'une nouvelle fonctionnalité ne doit pas nécessiter de modifier les écrans déjà existants. Chaque écran constitue un module indépendant qui peut être intégré au framework sans impacter le reste de l'architecture.

Pour garantir ces trois propriétés, le framework applique le principe de la **séparation des responsabilités**. Son organisation reflète directement la structure de l'application testée : chaque écran mobile possède son propre dossier, et chaque responsabilité technique est isolée dans un fichier dédié. Ainsi, les *locators* sont responsables de l'identification des éléments, les *screens* regroupent les actions réalisables sur ces éléments, tandis que le *Screen Manager* orchestre les interactions entre les différents écrans. Cette séparation rend le framework plus clair, plus robuste et beaucoup plus simple à maintenir sur le long terme.

3 Vue d'ensemble avant d'entrer dans les détails

Avant d'examiner le code en détail, il est utile d'avoir une vision globale de l'architecture du framework. Le schéma ci-dessous présente l'enchaînement complet des différents composants, depuis le scénario de test jusqu'au moteur d'automatisation MobileWright. Cette vue d'ensemble permettra de mieux comprendre le rôle de chaque élément avant d'étudier son implémentation.



Ce schéma met en évidence les différentes couches qui composent le framework ainsi que les relations entre elles. Dans les sections suivantes, chacune de ces couches sera étudiée progressivement, en partant du niveau le plus bas, qui regroupe les fonctionnalités techniques communes, jusqu'au niveau le plus élevé, où sont définis les scénarios de test lisibles et proches du langage naturel.

Pourquoi ? ne pas chercher à tout retenir ici ? Ce schéma est une carte mentale, pas un examen. Chaque nom de fichier et chaque flèche sera repris et expliqué en détail, un par un, dans les chapitres qui suivent.

4 Identifier les écrans et définir la structure du dépôt

Avant d'écrire la moindre ligne de code, la première étape consiste à identifier les **écrans** (ou *testwares*) que le framework devra représenter. En effet, le framework est conçu pour reproduire les interactions qu'un utilisateur effectue sur l'application. Chaque écran manipulé par les scénarios de test doit donc être modélisé par un composant logiciel dédié.

Dans ce projet, le scénario d'automatisation utilise deux écrans principaux : l'écran d'accueil du téléphone Android (**Home**) et l'application **YouTube**. Ces deux écrans constituent les premiers composants du framework.

Une fois ces écrans identifiés, leur organisation influence directement la structure du dépôt. Celui-ci est volontairement divisé en deux parties ayant des responsabilités bien distinctes :

- `src/` contient le **framework d'automatisation**, c'est-à-dire l'ensemble des composants réutilisables permettant d'interagir avec l'application.
- `tests/` contient les **scénarios de test**, qui exploitent le framework pour reproduire des parcours fonctionnels complets.

À l'intérieur du dossier `src/Screens/`, chaque écran est représenté par son propre dossier, dont le nom correspond à l'écran qu'il modélise.

```
src/
  Screens/
    screenBase.ts
    screenManager.ts
    Home/
      home.locator.ts
      home.screen.ts
    YouTube/
      youtube.locator.ts
      youtube.screen.ts
  fixtures.ts
tests/
  youTubeScenario.test.ts
```

Cette organisation n'a pas été choisie au hasard. On pourrait, par exemple, regrouper tous les écrans dans un seul fichier. Cependant, une telle approche rendrait rapidement le projet difficile à maintenir.

Pourquoi ? créer un dossier distinct pour chaque écran ?

Chaque écran possède son propre cycle de vie et évolue indépendamment des autres. Par exemple, une mise à jour de l'application YouTube peut modifier son interface sans qu'aucun changement ne soit apporté à l'écran d'accueil du téléphone. En regroupant tous les fichiers relatifs à un même écran dans un dossier dédié, les modifications restent localisées. Lorsqu'un écran évolue, un seul dossier est concerné, sans impacter le reste du framework. Cette organisation améliore considérablement la maintenabilité, la lisibilité et l'évolutivité du projet.

À l'intérieur de chaque dossier d'écran, deux fichiers — et seulement deux — sont créés.

Le premier, nommé `*.locator.ts`, décrit **les éléments présents sur l'écran**, c'est-à-dire les composants de l'interface que le framework doit être capable d'identifier (boutons, champs de recherche, vidéos, etc.).

Le second, nommé `*.screen.ts`, décrit **les actions réalisables sur cet écran**. Il regroupe toutes les interactions possibles avec les éléments définis dans le fichier des *locators*, comme cliquer sur un bouton, saisir un texte ou sélectionner une vidéo.

Cette séparation entre les **éléments de l'interface** (*locators*) et les **actions métier** (*screens*) constitue l'un des principes fondamentaux du framework. Elle simplifie la maintenance, favorise la réutilisation du code et servira de fil conducteur pour toutes les sections qui suivent.

5 ScreenBase : factoriser les fonctionnalités communes à tous les écrans

Avant de développer un écran spécifique, il est important d'identifier les fonctionnalités que tous les écrans ont en commun. En effet, certaines opérations ne dépendent d'aucun écran particulier : appuyer sur un élément, saisir du texte dans un champ, faire défiler l'écran ou encore effectuer un balayage (*swipe*) dans une direction donnée.

Si ces opérations étaient implémentées directement dans chaque classe d'écran, le même code serait dupliqué dans `HomeScreen`, `YouTubeScreen` et dans tous les futurs écrans du projet. Cette duplication rendrait le framework plus difficile à maintenir et augmenterait le risque d'erreurs. Pour éviter cela, toutes les fonctionnalités communes sont regroupées dans une classe de base, `ScreenBase`, située dans le fichier `src/Screens/screenBase.ts`. Les autres classes d'écran hériteront ensuite de cette classe afin de réutiliser automatiquement ces fonctionnalités. Nous détaillons ci-dessous, dans l'ordre où il est écrit, chaque décision de ce fichier.

```
import type { Locator, Screen } from '@mobilewright/core';

export class ScreenBase {
  protected readonly screen: Screen;

  constructor(screen: Screen) {
    this.screen = screen;
  }

  async clickOnApplicationIcon(locator: Locator): Promise<void> {
    await locator.tap();
  }

  async swipeUpFromScreenPoint(startX: number, startY: number,
      distance: number, duration?: number): Promise<void> {
    await this.screen.swipe('up', { startX, startY, distance, duration
        });
  }

  // ... autres gestes communs :
  // swipeDown, fillTextField, tap,
  // scroll, swipeLeft, swipeRight...
}
```

5.1 L'import : ne récupérer que des contrats de types

```
import type { Locator, Screen } from '@mobilewright/core';
```

IMPORT DE TYPE

Cette ligne importe uniquement deux **types** : `Screen` et `Locator`. Contrairement à un import classique, elle n'ajoute aucun code au programme final : elle sert exclusivement au compilateur TypeScript, afin de vérifier que les objets manipulés possèdent bien les méthodes attendues, telles que `tap()` ou `swipe()`. Une fois cette vérification effectuée, l'instruction disparaît lors de la génération du code JavaScript.

Pourquoi ? typer le paramètre `locator` avec `Locator` plutôt qu'avec `any` ? `any` désactiverait toute vérification du compilateur sur cette variable : n'importe quelle méthode pourrait être appelée dessus, même si elle n'existe pas, sans le moindre avertissement avant l'exécution. Ce serait incohérent avec le reste de ce chapitre, qui valorise justement le typage. En important le vrai type `Locator` fourni par `@mobilewright/core`, le compilateur vérifie que `locator.tap()` existe réellement, et le complète automatiquement dans l'éditeur.

5.2 Le champ protégé : visibilité et immutabilité

```
protected readonly screen: Screen
```

VISIBILITÉ ET IMMUTABILITÉ

Pourquoi ? choisir `protected` plutôt que `private` ou `public` ? Le mot-clé `protected` rend l'attribut accessible uniquement à l'intérieur de la classe `ScreenBase` et des classes qui en héritent, comme `HomeScreen` ou `YouTubeScreen` ; il reste inaccessible depuis le reste de l'application. C'est un compromis délibéré entre `private`, qui interdirait aussi l'accès aux classes filles, et `public`, qui rendrait l'attribut accessible depuis n'importe quel endroit du projet. Nous reviendrons sur cette comparaison de façon plus complète au chapitre consacré à `screenManager.ts`, où les trois niveaux de visibilité coexistent dans la même classe.

Le mot-clé `readonly` indique quant à lui que la référence contenue dans `screen` ne peut être affectée qu'une seule fois : une fois l'objet créé, elle ne pourra plus jamais être remplacée. Ce choix garantit que la connexion avec l'appareil mobile reste identique pendant toute l'exécution du test.

5.3 Le constructeur : recevoir la connexion plutôt que la fabriquer

```
constructor(screen: Screen) {
  this.screen = screen;
}
```

INJECTION DE DÉPENDANCES

Pourquoi ? recevoir `screen` en paramètre plutôt que le créer soi-même avec `new` ? Cette approche correspond au principe de l'**injection de dépendances** : la classe ne se préoccupe pas de savoir comment établir la connexion avec l'appareil, elle reçoit simplement une connexion déjà initialisée. Cette séparation des responsabilités améliore la flexibilité du framework et facilite la réalisation de tests unitaires, dans lesquels il est possible de remplacer un véritable appareil par un objet simulé (*mock*) reproduisant le même comportement.

5.4 L'héritage : ce que chaque écran reçoit gratuitement

HÉRITAGE

Les classes représentant les différents écrans, telles que `HomeScreen` ou `YouTubeScreen`, héritent de `ScreenBase` grâce au mot-clé `extends`. Ce mécanisme, appelé **héritage**, permet à chaque écran de réutiliser automatiquement toutes les fonctionnalités communes définies dans la classe de base, comme `clickOnApplicationIcon()` ou `swipeUpFromScreenPoint()`, sans avoir à les réimplémenter. Chaque classe d'écran peut ainsi se concentrer uniquement sur les interactions qui lui sont spécifiques, ce qui réduit la duplication du code et améliore la lisibilité ainsi que la maintenabilité du framework.

6 Les *Locators* : isoler la question « Où se trouve l'élément ? »

Une fois les fonctionnalités communes définies dans `ScreenBase`, la prochaine étape consiste à permettre au framework de localiser les éléments présents sur chaque écran. Avant de pouvoir cliquer sur un bouton, saisir un texte ou sélectionner une vidéo, il est indispensable de savoir où se trouve l'élément concerné dans l'interface. C'est précisément le rôle des fichiers `home.locator.ts` et `youtube.locator.ts` : décrire les éléments de l'interface utilisateur, sans effectuer la moindre action. Nous détaillons ci-dessous, ligne par ligne, le fichier `home.locator.ts`.

```
import type { Screen } from '@mobilewright/core';

export class HomeLocator {
  public readonly youTubeApplicationIconLocator;

  constructor(private readonly screen: Screen) {
    this.youTubeApplicationIconLocator =
      this.screen.getByRole('text', { name: 'YouTube' });
  }
}
```

6.1 La déclaration du champ : pourquoi `public`

```
public readonly youTubeApplicationIconLocator
```

VISIBILITÉ

Pourquoi ? rendre ce champ `public`, alors que le champ `screen` de `ScreenBase` est `protected` ? Contrairement à cette ressource interne qui doit rester protégée, un *locator* est conçu pour être utilisé par les classes d'écran, comme `HomeScreen`. Son unique rôle est de fournir un point d'accès vers un élément de l'interface : il n'existe donc aucune raison de le masquer avec `private`. Le rendre public simplifie son utilisation sans compromettre l'encapsulation.

6.2 Le constructeur : une syntaxe raccourcie propre à TypeScript

```
constructor(private readonly screen: Screen) {
    this.youTubeApplicationIconLocator =
      this.screen.getByRole('text', { name: 'YouTube' });
```

DÉCLARATION DE PARAMÈTRES

Cette écriture introduit une syntaxe appelée **déclaration de paramètres**. Au lieu de déclarer un attribut puis de l'initialiser manuellement dans le corps du constructeur, comme le fait `ScreenBase`, elle réalise les deux opérations en une seule instruction : le compilateur génère automatiquement un attribut nommé `screen`, lui applique les modificateurs `private` et `readonly`, puis l'initialise avec la valeur reçue en paramètre. Cette syntaxe est strictement équivalente à l'écriture longue, mais plus concise et largement utilisée dans les projets TypeScript.

6.3 Un choix assumé : l'initialisation immédiate du locator

LOCATOR EAGER

Pourquoi ? calculer `this.screen.getByRole(...)` une seule fois, dans le constructeur, plutôt qu'à chaque utilisation ? Cette stratégie est parfaitement adaptée au scénario de ce document, car la structure des écrans reste stable entre la création du `ScreenManager` et l'exécution des différentes actions. Elle atteint toutefois sa limite si l'application génère ou modifie dynamiquement son interface : dans ce cas, il serait préférable de calculer le *locator* à chaque utilisation, par exemple via une méthode ou un accesseur `get`, afin que le framework travaille toujours avec la version la plus récente de l'interface.

6.4 Le même principe, répété pour YouTube

Le fichier `youtube.locator.ts` applique exactement le même principe pour cinq éléments : l'icône de recherche, le champ de recherche, la vidéo, le bouton permettant d'ignorer la publicité et le curseur de progression de la vidéo. Aucune nouvelle règle de conception n'y est introduite ; cette répétition est volontaire, car appliquer systématiquement la même structure à chaque écran rend le framework plus homogène, plus prévisible et donc plus facile à comprendre, à maintenir et à faire évoluer.

7 Les classes *Screen* : transformer des opérations techniques en actions métier

Les fichiers `*.screen.ts` constituent la couche qui relie les fonctionnalités communes de `ScreenBase` aux éléments décrits dans les fichiers *locator*. Leur rôle n'est plus de localiser un élément ni d'exécuter un geste technique isolé, mais de composer ces briques pour offrir des actions métier, exprimées dans un langage proche de celui de

l'utilisateur. Autrement dit, une classe *Screen* répond à la question : *que souhaite faire l'utilisateur sur cet écran ?*

Le fichier `home.screen.ts` illustre parfaitement ce principe.

```
import { ScreenBase } from '../screenBase';
import type { Screen } from '@mobilewright/core';
import { HomeLocator } from './home.locator';

export class HomeScreen extends ScreenBase {
  private readonly homeLocator: HomeLocator;

  constructor(screen: Screen) {
    super(screen);
    this.homeLocator = new HomeLocator(screen);
  }

  async clickOnHomeButton(): Promise<void> {
    await this.screen.pressButton('HOME');
  }

  async clickOnYouTubeApplicationIcon(): Promise<void> {
    await this.clickOnApplicationIcon(
      this.homeLocator.youTubeApplicationIconLocator
    );
  }

  async middleSwipeUp(): Promise<void> {
    await this.swipeUpFromScreenPoint(238, 551, 1000);
  }

  async bottomSwipeUp(): Promise<void> {
    await this.swipeUpFromScreenPoint(224, 793, 1000);
  }
}
```

7.1 Trois imports, trois usages

IMPORT DE VALEUR OU IMPORT DE TYPE

Les trois instructions `import` du début du fichier n'ont pas la même finalité. `import { ScreenBase } from '../screenBase'` importe une véritable classe, dont `HomeScreen` a besoin pour hériter de ses méthodes via `extends`. `import type { Screen }` ne fait que reprendre, à titre de rappel, la vérification de types déjà expliquée au chapitre précédent : cet import disparaît entièrement au moment de la compilation. Enfin, `import { HomeLocator } from './home.locator'` importe elle aussi une classe réelle, puisqu'une instance en est créée un peu plus loin avec `new HomeLocator(screen)`.

7.2 L'héritage et le constructeur

HÉRITAGE

La ligne `extends ScreenBase` met en œuvre le mécanisme d'**héritage** : `HomeScreen` récupère automatiquement toutes les fonctionnalités communes définies dans `ScreenBase`, comme `clickOnApplicationIcon()` ou `swipeUpFromScreenPoint()`, sans avoir à les réécrire. Le constructeur commence obligatoirement par l'appel `super(screen)`, requis dès qu'une classe hérite d'une autre possédant elle-même un constructeur : cette instruction exécute d'abord celui de `ScreenBase`, qui initialise l'attribut `screen`, avant que la classe enfant ne puisse utiliser son propre objet `this` pour poursuivre son initialisation.

7.3 Le locator privé : un détail d'implémentation caché

ENCAPSULATION

Pourquoi ? déclarer `private readonly homeLocator: HomeLocator`, alors que les champs qu'il contient (comme `youTubeApplicationIconLocator`) sont `public` ? Contrairement aux *locators* qu'il contient, l'objet `homeLocator` est un détail d'implémentation propre à `HomeScreen`, que rien à l'extérieur n'a besoin de connaître. Une classe externe ne doit jamais manipuler les *locators* de `HomeScreen` directement, mais uniquement appeler les services qu'elle expose, comme `clickOnYouTubeApplicationIcon()`. Cette encapsulation limite les dépendances entre classes et améliore la maintenabilité du framework.

7.4 Des méthodes nommées par intention

Les méthodes de `HomeScreen` sont volontairement nommées selon une logique fonctionnelle plutôt que technique : `clickOnYouTubeApplicationIcon()`, `middleSwipeUp()` ou `bottomSwipeUp()` décrivent l'action réalisée par l'utilisateur, sans jamais révéler les détails de leur implémentation. C'est précisément ce vocabulaire qui permettra, plus loin, aux scénarios de test de se lire presque comme des phrases en langage naturel.

7.5 YouTubeScreen : le même principe, une nuance métier

Le fichier `youtube.screen.ts` applique exactement les mêmes principes architecturaux : héritage de `ScreenBase`, appel à `super()`, locator privé, méthodes nommées par intention. Une seule nuance mérite d'être signalée : `clickOnSkipAdButton()` vérifie d'abord si le bouton permettant d'ignorer la publicité est visible avant de cliquer, car cette publicité n'apparaît pas systématiquement et le test ne doit pas échouer en son absence. Ce comportement reste dans `YouTubeScreen`, et non dans `ScreenBase`, car il est spécifique à cet écran et n'a aucune raison d'être partagé avec le reste du framework.

Pourquoi ? la méthode `pressOkOnKeyboard()` tape-t-elle un point précis de l'écran plutôt que de simuler une touche « Entrée » ? Sur un écran natif, MobileWright ne propose pas d'équivalent à `press('Enter')` — ce mécanisme est réservé aux vues web (*WebView*). Sur une interface native, la validation du clavier passe donc par un geste réel : taper sur le bouton visible du clavier (ici, `OK`), exactement comme le ferait un utilisateur. Le nom de la méthode reflète fidèlement ce choix technique.

8 ScreenManager : le chef d'orchestre, expliqué ligne par ligne

Tous les éléments précédents — `ScreenBase`, les *locators* et les classes *Screen* — existent désormais de façon indépendante, mais aucun d'entre eux ne sait encore comment coexister. Il manque un seul élément : un point d'entrée unique que le scénario de test pourra appeler pour accéder à n'importe quel écran. C'est précisément le rôle du fichier `src/Screens/screenManager.ts`, le composant le plus dense en décisions de conception de tout le framework. Nous l'expliquons ici intégralement, dans l'ordre exact où il est écrit.

8.1 Les imports : trois lignes, trois natures différentes

```
import { HomeScreen } from './Home/home.screen';
import { YouTubeScreen } from './YouTube/youtube.screen';
import type { Screen } from '@mobilewright/core';
```

IMPORT DE VALEUR OU IMPORT DE TYPE

Les deux premières lignes importent des **classes réelles**. Une classe est à la fois un type (elle définit une forme) et une valeur (elle existe au moment de l'exécution, ce qui permet d'écrire `new HomeScreen(...)`). C'est pour cette raison qu'elles sont importées sans le mot-clé `type` : le code compilé a réellement besoin de la classe elle-même pour pouvoir l'instancier un peu plus loin.

La troisième ligne, `import type { Screen }`, n'importe qu'un contrat de types fourni par le cœur de MobileWright (`@mobilewright/core`). Le type `Screen` n'est jamais instancié directement dans ce fichier — l'instruction `new Screen()` n'apparaît jamais — il sert uniquement à indiquer au compilateur que le paramètre reçu respecte bien cette forme. Comme ce type disparaît entièrement à la compilation, l'écrire avec `import type` évite d'alourdir inutilement le code final.

8.2 La déclaration de la classe et son export

```
export class ScreenManager {
  ...
}
```

Pourquoi ? exporter précisément cette classe ? Parce que `ScreenManager` est l'unique point d'entrée que les fichiers extérieurs au framework, comme `fixtures.ts`, ont besoin d'importer. `HomeScreen` et `YouTubeScreen` sont eux aussi exportés depuis leurs propres fichiers, mais en pratique, aucun scénario de test n'a besoin de les importer directement : on y accède toujours à travers le manager. Le mot-clé `export` est le mécanisme qui rend une classe visible en dehors de son propre fichier ; sans lui, `ScreenManager` resterait enfermé dans `screenManager.ts` et serait totalement inutilisable ailleurs.

8.3 Les champs de la classe : la visibilité expliquée par un contraste réel

```
private readonly screen: Screen;
public readonly homeScreen: HomeScreen;
public readonly youTubeScreen: YouTubeScreen;
```

ENCAPSULATION

Ces trois champs sont déclarés ici mais ne reçoivent aucune valeur : leur assignation aura lieu dans le constructeur, juste après. C'est le moment idéal pour répondre précisément à une question centrale : pourquoi `screen` est-il `private` alors que `homeScreen` et `youTubeScreen` sont `public` ?

`screen` représente la connexion brute à l'appareil, telle que fournie par MobileWright. Un scénario de test n'a jamais besoin d'appeler directement des méthodes bas niveau sur cette connexion : cela reviendrait à contourner tout le travail réalisé par `HomeScreen` et `YouTubeScreen`, et à réintroduire dans les tests exactement la complexité technique que le framework existe pour masquer. Ce champ est donc `private` : strictement invisible et inutilisable en dehors de `ScreenManager` lui-même. C'est ici que se manifeste l'**encapsulation** : cacher un détail d'implémentation qui n'a aucune raison de fuiter vers l'extérieur.

`homeScreen` et `youTubeScreen` représentent au contraire exactement ce que le scénario de test doit pouvoir utiliser : les actions métier de chaque écran. Les rendre `public` n'est donc pas accessoire, mais nécessaire — sans cette visibilité, aucun scénario ne pourrait jamais appeler `clickOnHomeButton()` ou `fillSearchField()`.

`readonly` s'applique aux trois champs pour la même raison qu'expliquée au chapitre consacré à `ScreenBase` : une fois le manager construit, ni la connexion à l'appareil, ni les instances d'écran, ne doivent pouvoir être remplacées en cours de route. Un scénario ne doit jamais pouvoir écrire accidentellement `screenManager.homeScreen = autreChose`.

8.4 Le constructeur : le moment où trois objets prennent vie

```
constructor(screen: Screen) {
  this.screen = screen;
  this.homeScreen = new HomeScreen(this.screen);
  this.youTubeScreen = new YouTubeScreen(this.screen);
}
```

INJECTION DE DÉPENDANCES

Pourquoi ? recevoir `screen` en paramètre plutôt que le créer soi-même à l'intérieur du constructeur ? Parce que la connexion à un appareil mobile est fournie par le moteur MobileWright au moment où un test démarre. `ScreenManager` n'a ni besoin de savoir, ni le droit de décider, comment cette connexion est établie — quel appareil, quel émulateur, quelle configuration. Il se contente de *recevoir* une connexion déjà

prête et de la faire circuler. C'est le principe d'**injection de dépendances** : une classe reçoit ce dont elle dépend au lieu de le fabriquer elle-même, ce qui la rend plus simple, plus prévisible et plus facile à tester isolément.

La première ligne, `this.screen = screen`, ne crée aucun nouvel objet — il n'y a pas de `new` — elle copie simplement la référence reçue dans le champ privé. Les deux lignes suivantes sont différentes : `new HomeScreen(this.screen)` et `new YouTubeScreen(this.screen)` fabriquent réellement, pour la première fois dans l'exécution du programme, les deux objets qui porteront toute la logique métier des écrans. Ce sont les deux seuls endroits de toute l'application où ces classes sont instanciées : `ScreenManager` est le point unique de création, ce qui garantit qu'il n'existe jamais deux instances concurrentes et incohérentes de `HomeScreen`.

Pourquoi ? transmettre `this.screen` — et non un nouvel objet `screen` — à chaque écran ? Parce que `HomeScreen` et `YouTubeScreen` doivent piloter le *même* appareil que celui reçu par le manager. C'est ici de la **composition** : `ScreenManager` ne devient pas un `HomeScreen`, il en possède un, construit à partir de la même dépendance que celle qu'il a lui-même reçue.

8.5 Les méthodes d'accès : pourquoi retourner plutôt que recréer

```
onHomeScreen() {
  return this.homeScreen;
}

onYouTubeScreen() {
  return this.youTubeScreen;
}
```

ABSTRACTION

Ces deux méthodes ne font rien d'autre que renvoyer une référence déjà existante — aucun `new` n'apparaît ici. C'est volontaire, et important : les instances de `HomeScreen` et `YouTubeScreen` ont déjà été créées une seule fois, dans le constructeur, à l'étape précédente. `onHomeScreen()` et `onYouTubeScreen()` ne sont que des *portes d'accès* vers ces objets déjà prêts, jamais des usines à nouvelles instances. Si elles recréaient un nouvel objet à chaque appel, deux appels successifs à `onHomeScreen()` dans un même test pourraient renvoyer deux objets différents, ce qui romprait toute cohérence d'état entre les étapes du scénario.

C'est également ici qu'apparaît l'**abstraction** : le scénario de test qui écrit `screenManager.onHomeScreen().clickOnHomeButton()` n'a besoin de rien savoir sur la manière dont `HomeScreen` a été construit, ni sur l'existence du champ privé `screen`. Il se contente d'appeler une méthode simple, et tout le reste — l'orchestration, l'injection de dépendances, la visibilité des champs — reste caché derrière cette façade.

Pourquoi ? ces méthodes s'appellent-elles `onHomeScreen()` et `onYouTubeScreen()`, avec une minuscule initiale ? La convention TypeScript réserve la majuscule initiale

aux noms de classes et de types ; une méthode commence toujours par une minuscule. Respecter cette règle partout évite toute ambiguïté visuelle entre « ceci est une classe » et « ceci est une action à appeler ».

9 Fixtures : brancher le framework sur le moteur de test

Le fichier `src/fixtures.ts` constitue le pont entre `ScreenManager` — le framework que nous venons de construire — et les fichiers de scénario du dossier `tests/`. Nous l'expliquons ci-dessous dans l'ordre où il est écrit.

```
import { test as base } from '@mobilewright/test';
import { ScreenManager } from './Screens/screenManager';

type Fixtures = {
  screenManager: ScreenManager;
};

export const test = base.extend<Fixtures>({
  screenManager: async ({ screen }, use) => {
    const screenManager = new ScreenManager(screen);
    await use(screenManager);
  },
});

export { expect } from '@mobilewright/test';
```

9.1 Le type `Fixtures` : décrire une ressource avant de la fournir

```
type Fixtures = {
  screenManager: ScreenManager;
};
```

CONTRAT DE TYPES

Ce type ne crée rien : il décrit, pour le compilateur, la forme exacte de la nouvelle ressource que ce fichier va ajouter au système de test — une propriété nommée `screenManager`, de type `ScreenManager`. Grâce à cette déclaration, chaque test qui écrira `async ({ screenManager })=> {...}` bénéficiera de l'auto-complétion et de la vérification de types sur `screenManager`, exactement comme sur n'importe quelle autre variable.

9.2 Étendre le système de test de base

EXTENSION DE FIXTURE

L'appel `base.extend<Fixtures>(...)` étend le système de test fourni par MobileWright pour lui ajouter une nouvelle ressource nommée `screenManager`, disponible

automatiquement dans chaque test. Le type `Fixtures` déclaré juste avant sert uniquement à décrire, pour le compilateur, la forme de cette nouvelle ressource.

Pourquoi ? passer par `base.extend(...)` plutôt que de créer un `ScreenManager` directement dans chaque fichier de scénario ? Parce que cela obligerait chaque scénario à connaître les détails de construction du manager, exactement le genre de détail que ce framework cherche à cacher. En centralisant cette construction dans `fixtures.ts`, tous les scénarios du projet obtiennent un `screenManager` prêt à l'emploi, de la même manière, sans dupliquer ce code.

9.3 Ce que fait exactement la fonction fournie

UNE CRÉATION PAR TEST

La fonction fournie reçoit déjà `screen` — la connexion à l'appareil, préparée par MobileWright avant même que ce code ne s'exécute —, crée *exactement une fois par test* un `new ScreenManager(screen)`, puis le transmet via `use(screenManager)`.

9.4 Le rôle précis de `use(...)`

POINT DE REMISE

Pourquoi ? appeler `use(screenManager)` plutôt que de simplement faire `return screenManager` ? `use(...)` n'est pas un retour classique : c'est le point précis où le moteur de test MobileWright reprend la main, exécute le corps du test en lui fournissant le `screenManager` construit, puis revient *après* cet appel une fois le test terminé. C'est ce qui permettrait, si nécessaire, d'ajouter du code de nettoyage juste après la ligne `await use(screenManager)` — par exemple fermer une ressource — sans jamais toucher aux fichiers de scénario. Dans ce projet, aucun nettoyage n'est nécessaire, mais la structure du mécanisme reste la même.

Pourquoi ? est-ce précisément ici, et nulle part ailleurs, que la connexion `screen` rencontre `ScreenManager` ? Parce que c'est le rôle de `fixtures.ts` d'assurer cette liaison, une seule fois, au même endroit, pour tous les scénarios du projet. Le manager lui-même, comme expliqué au chapitre précédent, ne sait pas d'où vient `screen` : ce n'est pas son rôle de le savoir.

10 Le scénario : la preuve concrète du bénéfice

Tout ce travail de construction n'a de sens que s'il se traduit par un scénario de test simple à lire, même pour une personne qui ne programme pas. Voici le fichier `tests/youTubeScenario.test.ts` tel qu'il est écrit dans ce projet.

```
import { test, expect } from '../src/fixtures';

test('YouTube scenario', async ({ screenManager }) => {
  await screenManager.onHomeScreen().clickOnHomeButton();
```

```
    await screenManager.onHomeScreen().middleSwipeUp();
    await screenManager.onHomeScreen().clickOnYouTubeApplicationIcon();
    await screenManager.onHomeScreen().bottomSwipeUp();
    await screenManager.onHomeScreen().middleSwipeUp();
    await screenManager.onHomeScreen().clickOnYouTubeApplicationIcon();
    await screenManager.onYouTubeScreen().clickOnSearchIcon();
    await screenManager.onYouTubeScreen().fillSearchField('MobileWright'
        );
    await screenManager.onYouTubeScreen().pressOkOnKeyboard();
    await screenManager.onYouTubeScreen().clickOnVideo();
    await screenManager.onYouTubeScreen().clickOnSkipAdButton();
    await screenManager.onYouTubeScreen().swipeVideoProgressSliderRight
        (1000);
});
```

10.1 Ce que ce fichier ne fait jamais

RIEN À CONSTRUIRE ICI

La signature `async ({ screenManager })=>` reçoit directement l'objet préparé par `fixtures.ts` : aucune ligne de ce fichier ne crée de connexion, ne configure d'appareil, ni ne construit d'objet — tout cela a déjà été réalisé en amont.

10.2 Ce que ce fichier raconte

LE MÊME SCHÉMA, RÉPÉTÉ

Le test se contente de raconter, dans l'ordre, l'histoire de l'utilisateur : revenir à l'accueil, faire défiler l'écran, ouvrir YouTube, chercher une vidéo, l'ouvrir, ignorer la publicité si elle apparaît, puis avancer dans la lecture. Chaque ligne suit exactement le même schéma, celui expliqué en détail au chapitre précédent : `screenManager` donne accès à l'écran (`onHomeScreen()` ou `onYouTubeScreen()`), qui donne lui-même accès à une action métier déjà nommée de façon compréhensible.

10.3 La preuve de maintenabilité

Pourquoi ? ce fichier ne bougera-t-il jamais, même si l'application change ? Si demain l'identifiant du champ de recherche change dans l'application YouTube, une seule ligne devra être modifiée — dans `youtube.locator.ts` — et ce fichier de scénario restera intact. C'est exactement le bénéfice recherché depuis le premier chapitre de ce document.

11 En résumé

Le framework construit dans ce document repose sur une idée simple derrière chaque décision technique : chaque responsabilité possède un seul endroit où vivre. Cibler

un élément vit dans un *locator*. Agir sur un écran vit dans une classe *Screen*. Les gestes partagés entre écrans vivent dans `ScreenBase`. Le choix de l'écran actif vit dans `ScreenManager`. Le branchement au moteur de test vit dans `fixtures.ts`. Rien de tout cela n'est arbitraire : chaque visibilité — `private`, `protected` ou `public` —, chaque `export`, chaque instanciation avec `new`, répond à une question précise sur qui a besoin de savoir quoi. C'est cette discipline qui permet au scénario final de rester lisible en une phrase.